

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizamos los nombres para contextualizar el problema
nombres <- c("Camilo", "Andrés", "Sofía", "Manuel", "Laura", "Carlos",
             "Daniela", "Miguel", "Valentina", "Eduardo", "Natalia",
             "José", "Isabella", "Gabriel", "Mariana")
nombre <- sample(nombres, 1)

# Aleatorizamos los tipos de recipientes
recipientes <- c("cilindro", "tubo", "conducto", "tanque cilíndrico",
                "recipiente cilíndrico", "contenedor cilíndrico")
recipiente <- sample(recipientes, 1)

# Aleatorizar adjetivos para el cilindro interno
adjetivos_interno <- c("interno", "hueco", "vacío", "interior")
adjetivo_interno <- sample(adjetivos_interno, 1)

# Aleatorizar líquidos
liquidos <- c("aceite", "combustible", "líquido", "fluido")
liquido <- sample(liquidos, 1)

# Aleatorizar lo que se desea calcular
calculos <- c("la cantidad de", "el volumen de", "cuánto", "qué cantidad de")
calculo <- sample(calculos, 1)

# Aleatorizar medidas del cilindro (manteniendo consistencia matemática)
# Diámetro externo (entre 0.3 y 0.8 metros)
d_ext <- round(runif(1, 0.3, 0.8), 1)

# Radio interno (entre 0.5 y 1.5 metros)
r_int <- round(runif(1, 0.5, 1.5), 1)

# Altura del cilindro (entre 1 y 3 metros)
altura <- round(runif(1, 1, 3), 1)

# Asegurar que las dimensiones tengan sentido (radio interno < radio externo)
while (r_int >= d_ext/2) {
  d_ext <- round(runif(1, max(r_int * 2.1, 0.3), max(r_int * 3, 0.8)), 1)
}

# Calcular radio externo
r_ext <- d_ext / 2

# Calcular el volumen necesario (es el volumen del cilindro interior)
volumen <- pi * (r_int^2) * altura

# Verificar dimensiones para coherencia física
# El diámetro externo debe ser mayor que dos veces el radio interno
if (d_ext <= 2 * r_int) {
  d_ext <- 2 * r_int + round(runif(1, 0.1, 0.5), 1)
}

```

```

  r_ext <- d_ext / 2
}

# Aleatorizar unidades de medida
unidades_longitud <- c("m", "cm", "dm")
unidad <- sample(unidades_longitud, 1)

# Ajustar valores según la unidad
factor_conversion <- 1
if (unidad == "cm") {
  factor_conversion <- 100
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
} else if (unidad == "dm") {
  factor_conversion <- 10
  d_ext <- d_ext * factor_conversion
  r_int <- r_int * factor_conversion
  altura <- altura * factor_conversion
  volumen <- volumen * (factor_conversion^3)
  r_ext <- r_ext * factor_conversion
}

# Redondear todos los valores para mayor claridad
d_ext <- round(d_ext, 1)
r_int <- round(r_int, 1)
altura <- round(altura, 1)
r_ext <- round(r_ext, 1)
volumen <- round(volumen, 1)

# La respuesta correcta es "C) Altura del cilindro"
# Ya que para calcular el volumen necesitaríamos conocer la altura
opcion_correcta <- 3 # Índice de "Altura del cilindro" en el vector de opciones

# Vector de solución para r-exams (1 para la correcta, 0 para las incorrectas)
solucion <- c(0, 0, 1, 0)

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar el cilindro
codigo_python <- paste0("
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Ellipse, Rectangle, Arrow, FancyArrowPatch

# Parámetros del cilindro
radio_interno = ", r_int, " # ", unidad, "
grosor = ", r_ext - r_int, " # ", unidad, "
radio_externo = radio_interno + grosor # ", r_ext, " ", unidad, "
altura = ", altura, " # ", unidad, "
diametro_externo = 2 * radio_externo # ", d_ext, " ", unidad, "
unidad = '", unidad, "'

```

```

# Crear figura y ejes - reducida en un 25%
fig, ax = plt.subplots(figsize=(7.5, 9)) # Reducido de (10, 12) a (7.5, 9)

# Ajustar límites del gráfico
ax.set_xlim(-radio_externo*1.5, radio_externo*1.5)
ax.set_ylim(-altura*0.2, altura*1.4)
ax.axis('equal')
ax.axis('off')

# Factor de compresión para la perspectiva
factor_compresion = 0.4

# Dibujar el cilindro externo
# Base inferior
ax.add_patch(Ellipse((0, 0), width=2*radio_externo, height=2*radio_externo*factor_compresion,
                    fill=False, edgecolor='black', linestyle='-'))

# Base superior
ax.add_patch(Ellipse((0, altura), width=2*radio_externo, height=2*radio_externo*factor_compresion,
                    fill=False, edgecolor='black', linestyle='-'))

# Laterales externos
ax.plot([-radio_externo, -radio_externo], [0, altura], 'k-')
ax.plot([radio_externo, radio_externo], [0, altura], 'k-')

# Dibujar el cilindro interno
# Base inferior interna (línea punteada)
ax.add_patch(Ellipse((0, 0), width=2*radio_interno, height=2*radio_interno*factor_compresion,
                    fill=False, edgecolor='black', linestyle='--'))

# Base superior interna (línea punteada)
ax.add_patch(Ellipse((0, altura), width=2*radio_interno, height=2*radio_interno*factor_compresion,
                    fill=False, edgecolor='black', linestyle='--'))

# Laterales internos (líneas punteadas)
ax.plot([-radio_interno, -radio_interno], [0, altura], 'k--')
ax.plot([radio_interno, radio_interno], [0, altura], 'k--')

# Añadir etiquetas con flechas - corregidas para terminar exactamente en los límites
# Diámetro externo en la parte superior
ax.annotate(f'Diámetro externo = {diametro_externo} {unidad}',
           xy=(0, altura + 0.15*altura),
           xytext=(0, altura + 0.15*altura),
           ha='center', va='center', fontsize=9)

# Usar annotate con arrowprops para flechas precisas del diámetro externo
ax.annotate('',
           xy=(-radio_externo, altura + 0.1*altura), # Punto exacto donde termina la flecha
           xytext=(radio_externo, altura + 0.1*altura), # Punto de inicio
           arrowprops=dict(arrowstyle='<->', color='black', lw=1))

# Radio interno - flecha precisa
ax.annotate(f'Radio interno = {radio_interno} {unidad}',

```

```

        xy=(radio_interno/2, altura + 0.07*altura),
        xytext=(radio_interno/2, altura + 0.07*altura),
        ha='center', va='center', fontsize=9)
ax.annotate('',
        xy=(radio_interno, altura), # Punto exacto donde termina la flecha
        xytext=(0, altura), # Punto de inicio
        arrowprops=dict(arrowstyle='->', color='black', lw=1))

# Grosor - Reposicionado para mejor visibilidad
ax.annotate(f'Grosor = {grosor} {unidad}',
        xy=(-(radio_interno + grosor/2), altura/2), # Punto medio entre radio interno y externo
        xytext=(-radio_externo - 0.3*radio_externo, altura/2),
        ha='right', va='center', fontsize=9,
        arrowprops=dict(arrowstyle='->', color='black', lw=1))

# Línea horizontal para mostrar el grosor - engrosada para mayor visibilidad
ax.plot([-radio_externo, -radio_interno], [altura/2, altura/2], 'k-', lw=3.0) # Línea más gruesa
# Marcas verticales más visibles
ax.plot([-radio_externo, -radio_externo], [altura/2-0.07*altura, altura/2+0.07*altura], 'k-', lw=2.5)
ax.plot([-radio_interno, -radio_interno], [altura/2-0.07*altura, altura/2+0.07*altura], 'k-', lw=2.5)

# Añadir sombreado para destacar el grosor
import matplotlib.patches as patches
rect = patches.Rectangle(
    (-radio_externo, altura/2-0.03*altura), # Posición (x,y) de la esquina inferior izquierda
    grosor, # Ancho
    0.06*altura, # Alto
    linewidth=0,
    color='lightgray',
    alpha=0.5
)
ax.add_patch(rect)

# Altura - flecha precisa con etiqueta mejorada
# Texto de altura separado y más visible
ax.text(radio_externo + 0.3*radio_externo, altura/2, 'Altura',
        ha='center', va='center', fontsize=10, fontweight='bold')

# Flecha bidireccional para la altura
ax.annotate('',
        xy=(radio_externo + 0.1*radio_externo, altura), # Punto exacto donde termina la flecha arriba
        xytext=(radio_externo + 0.1*radio_externo, 0), # Punto exacto donde termina la flecha abajo
        arrowprops=dict(arrowstyle='<->', color='black', lw=1.5))

# Líneas horizontales para marcar los límites de la altura
ax.plot([radio_externo, radio_externo + 0.2*radio_externo], [0, 0], 'k-', lw=1)
ax.plot([radio_externo, radio_externo + 0.2*radio_externo], [altura, altura], 'k-', lw=1)

# Radio externo - flecha precisa
ax.annotate(f'Radio externo = {radio_externo} {unidad}',
        xy=(radio_interno, -0.1*altura),
        xytext=(radio_interno, -0.05*altura),
        ha='center', va='center', fontsize=9)

```

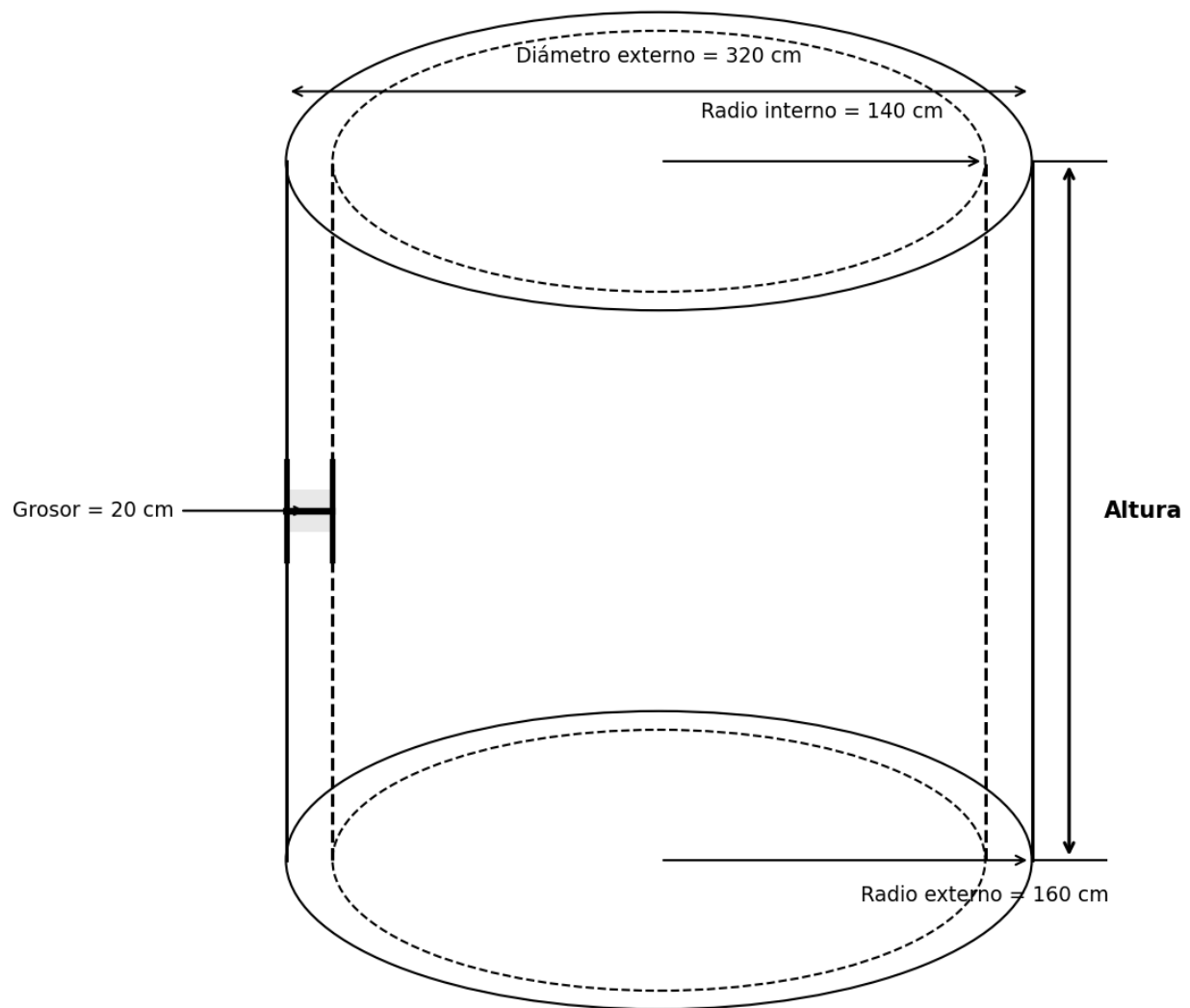
```
ax.annotate('',
            xy=(radio_externo, 0), # Punto exacto donde termina la flecha
            xytext=(0, 0), # Punto de inicio
            arrowprops=dict(arrowstyle='->', color='black', lw=1))

plt.tight_layout()
plt.savefig('cilindro_hueco.png', dpi=150, bbox_inches='tight')
plt.close()
")

# Ejecutar código Python para generar la figura
py_run_string(codigo_python)
```

Question

Manuel desea saber cuánto fluido se necesita para llenar el tubo interno, pero solamente cuenta con las medidas de las dimensiones que muestra la figura.



¿Cuál medida le falta a Manuel para hallar la cantidad deseada?

Answerlist

- Radio externo.
- Diámetro externo.

- Altura del cilindro.
- Perímetro del cilindro.

Solution

La respuesta correcta es: **Altura del cilindro.**

Para calcular el volumen de fluido necesario para llenar el tubo hueco, necesitamos calcular el volumen de un cilindro, cuya fórmula es:

$$V = \pi \cdot r^2 \cdot h$$

Donde: - r es el radio (en este caso, el radio interno = 140 cm) - h es la altura del cilindro

En el problema se nos proporciona: - Diámetro externo = 320 cm - Radio interno = 140 cm

Sin embargo, no se nos proporciona la altura del cilindro, que es esencial para calcular el volumen. Por lo tanto, a Manuel le falta conocer la altura del cilindro para poder calcular la cantidad de fluido necesaria.

Si tuviéramos la altura, el volumen se calcularía así: $V = \pi \cdot (140)^2 \cdot h = 61575,22 \cdot h \text{ cm}^3$

Answerlist

- Falso
- Falso
- Verdadero
- Falso

Meta-information

exname: volumen_cilindro_hueco extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría|Volumen|Cilindro

Metadatos ICFES

exextra[Tópico]: Geometría exextra[Competencia]: Razonamiento exextra[Dificultad]: Media exextra[Estandar]: Utilizo técnicas y herramientas para la construcción de figuras planas y cuerpos con medidas dadas. exextra[Aprendizaje]: Establecer y utilizar diferentes procedimientos de cálculo para hallar medidas de superficies y volúmenes. exextra[Evidencia]: Resolver problemas que involucran el cálculo de volúmenes de cilindros. exextra[Componente]: Espacial-Métrico exextra[Proceso]: Resolución de problemas exextra[Contexto]: Científico exextra[Estrategia]: Identificar los datos faltantes para resolver un problema de volumen de un cilindro hueco. exextra[Tiempo]: 3 minutos exextra[Pregunta]: 1 exextra[Aleatorización]: Si exextra[Esquema]: Selección múltiple con única respuesta exextra[Formato]: Imagen